

概率、条件概率及贝叶斯公式



有时候，对于数据集进行简单可视化处理（比如散点图、直方图）或做一些简单的代数计算（比如求平均值）就能让我们大致了解数据中所含有的信息。但在大多数情况下，我们希望更进一步，从数据中挖掘出更一般性、更为深刻的结论。比如，我们希望探索数据是如何生成的，或根据以往数据对未来的结果做出预测，又或是从对数据总体的多个猜测中找出最合理的一个。这些正是统计推断所关注的内容。

统计推断的基础是随机性 (randomness)。随机性是偶然性的一种形式，但有些细微区别。随机性用来描述某事件集中的各个事件是否发生的不确定性，例如某次试验得到的观察值的不确定性。但是，这种结果还是有规律可循的。比如，抛一枚硬币，可能会出现正面朝上或者反面朝上两种结果。至于出现哪种结果，是随机的，但是，试验结果一定是这两种中的某一种结果；如果硬币是均匀的，则正面朝上或者反面朝上两种结果的频率会大体差不多。至于偶然性，更多的是描述完全预料之外的结果。比如，抛一枚硬币，但这枚硬币抛到空中后没有掉下来，完全找不到了。这就是预料之外的结果。

对随机性的度量，我们用概率（probability）定义，反映某事件发生可能性的大小。在概率的基础上，我们将其延伸到条件概率（conditional probability）的定义，即若事件 A 已经发生，事件 B 发生的概率。本章中，我们将引入概率、条件概率和他们密切相关的概率分布、随机变量等的定义，详细介绍计算条件概率的贝叶斯公式（Bayes rule），并通过若干案例展现它们在实际中的应用。

概率是对一个事件发生可能性的度量，记作 P 或者 Pr 。比如，“如果今年我在北京地区参加高考的话，我能够考上中国人民大学”就是一个随机事件。因为这个事件是不一定发生的，是带有随机性的，考得上与考不上均有可能发生。我们可以用符号 A 来表示该随机事件：

$$A = \{\text{如果今年我在北京地区参加高考的话，我能够考上中国人民大学}\}.$$

但是我们可以讨论这个随机事件发生的概率。

随机事件 A 发生的概率为 $\Pr(A)$ ，表示“我能够考上中国人民大学”的概率。比方说，大家可以认为

$$\Pr(\text{如果今年我在北京地区参加高考的话, 我能够考上中国人民大学}) = 0.5.$$

概率可以是一个主观感受。上面就是一个主观概率的例子。概率的取值总在 0 和 1 之间，概率的取值越大，表明该随机事件发生的概率越大。发生概率为 0 的事件，一般称为 0 概率事件。但是，0 概率事件不是不可能发生的事件，0 概率事件是可能发生的。同样的，发生概率为 1 的事件，不一定是必然发生的事件。但在这个章节中，我们可以忽略这个 0 概率事件和不可能事件 (类似的，1 概率事件与必然事件之间) 的区别。

概率可以是一个客观计算的结果。举一个投骰子的例子，由于正常的骰子是中心对称的正方体，那么随机抛掷出现每个面 $\{1, 2, 3, 4, 5, 6\}$ 朝上的可能性均等，且一定不可能出现其他数字。因此，出现 $\{1, 2, 3, 4, 5, 6\}$ 中任一点数的概率为 $1/6$ ，可以记作为 $\Pr(\text{出现点数 } k) = 1/6$ ，其中 $k = 1, 2, 3, 4, 5, 6$ 。

在数学上，可以证明无限循环小数 $0.9999 \cdots$ 和 1 是相等的，即 $0.9999 \cdots = 1$ 。但是，如果不是无限循环而只有有限位的话，比如 0.9999 ，那就和 1 很不一样了。辛普森案件就是一个例子，其中就提到了 $0.9999 \neq 1$ 。

1994 年，前美式黑人橄榄球运动员辛普森 (O.J. Simpson) 杀妻一案成为当时美国最为轰动的事件。此案当时的审理一波三折，辛普森在用刀杀前妻及餐馆的侍应生郎·高曼两项一级谋杀罪的指控中，由于警方的几个重大失误导致有力的证据失效，最终以无罪获释，仅被民事判定为对两人的死亡负有责任。本案也成为美国历史上疑罪从无的最大案件。辛普森既是美国橄榄球明星，也是演员，在美国是家喻户晓的人物，这个案件是要判定他是否杀了前妻和前妻男友，被称为“世纪审判”。在审判的过程中，审判团组成成员确定为 9 个黑人、2 个白人、1 个拉美人，也即有四分之三都是黑人。1995 年 1 月正式开始法庭辩论，检方的证据主要是：有作案动机；曾经虐待前妻；不满意前妻和其男友；经过 DNA 检验，血手套 DNA 相似的概率为 0.9999。

法庭辩论中，辛普森的律师指出以下几点：

- 血手套是几年前前妻买给辛普森的，但是辛普森戴不进去，最后很困难的戴进去了；
- 警察抽了辛普森 7 毫升血，最后只剩 5.5 毫升，有 1.5 毫升不知去向；
- 没有直接目击者，也一直没有找到凶器；
- 辛普森两个月前买过一把猎刀，但是上面没有找到任何血迹。

最后辛普森的律师认为发现血手套的警察仇视黑人，少的 1.5 毫升血液可能是被这个警察为了栽赃辛普森故意滴到血手套上了。同时从作案时间到飞机起飞仅有半个小时，杀人后洗清血迹，然后再登机，时间并不很充裕。美国有一句话：“一碗面条里只有一条虫子。”（Only a bowl of noodles with a worm）意思是碗里只要找到一条虫子就不能吃了，再去找第二条在法律上是没意义的。

这个案子，检方开始是很顺利的，他们又提出了许多多余的证据，结果弄巧成拙。除了检方证据出现的漏洞外，当时处于黑人受到压制的年代，考虑到陪审团大多是黑人，辛普森的律师团队大打同情牌，这更加让陪审团的支持向辛普森倾斜。最终在 1995 年 10 月 3 日，12 位陪审团成员一致认为辛普森无罪，因为已有的证据不足以认定有罪，只能做无罪推定。所有黑人欢呼雀跃，但大多白人非常失望，并开始质疑陪审团制度和 DNA 检测。刑事审判结束之后，在 1997 年，法庭认定辛普森对虐待前妻以及其他一些行为要负民事责任，2 月 10 日，判定辛普森赔偿 3350 万美元给他的前妻及前妻的男友。

这个案例并没有直接运用统计方法，只是我们可以注意几个细节，辛普森的律师曾经问 DNA 检验专家，0.9999 的概率是否就意味着辛普森一定是有罪的，专家只能说“NO”。法庭哗然。这表明，概率在法庭上只是一个可能性，只要不是 100%，就不能仅依此一项证据定罪。这个案件没有直接的证人，没有直接的凶器，缺少最最重要的证据，无法排除所有可能的合理嫌疑，因此只能做无罪推定。

概率的性质

全事件指所有可能出现结果的全体，常用 Ω 表示，因此全事件发生的概率一定是 1，即 $\Pr(\Omega) = 1$ 。空事件指没有任何结果出现的事件，这通常是不可能发生的，所以 $\Pr(\emptyset) = 0$ 。对于 N 个互不相交的事件 $A_1, A_2, \dots, A_N$ ，概率具有可加性，即

$$\Pr(A_1 \cup A_2 \cup \dots \cup A_N) = \sum_{i=1}^N \Pr(A_i).$$

这个可加性质对于无穷可数的互不相交的事件仍然成立。根据概率的可加性，一个简单的推论是，对于两个不相交的事件 A 和 B ，我们有 $\Pr(A \cup B) = \Pr(A) + \Pr(B)$ 。对于掷子的例子，

- $\Pr(\text{出现点数 } 100) = 0$
- $\Pr(\text{出现点数小于 } 7 \text{ 大于 } 0) = 1$
- $\Pr(\text{出现点数为 } 2 \text{ 或者 } 3) = \Pr(\text{出现点数 } 2) + \Pr(\text{出现点数 } 3) = 1/3$

条件概率指的是，对于两个事件 A 和 B ，若事件 B 已经发生，事件 A 发生的概率大小。我们将这样的条件概率记作 $\Pr(A \mid B)$ 。在数学上，条件概率的定义依赖于概率，其公式为

$$\Pr(A \mid B) = \frac{\Pr(A \cap B)}{\Pr(B)} \quad (1)$$

其中 $A \mid B$ 表示在事件 B 发生的情况下的事件 A ， $A \cap B$ （也可以写成 AB ）表示事件 A 和 B 的交，也就是事件 A 和 B 同时发生， $\Pr(A \cap B)$ 也被称为联合概率。因此，这个公式描述了，在事件 B 发生的情况下发生事件 A 的概率等于事件 A 和 B 同时发生的概率除以事件 B 发生的概率。

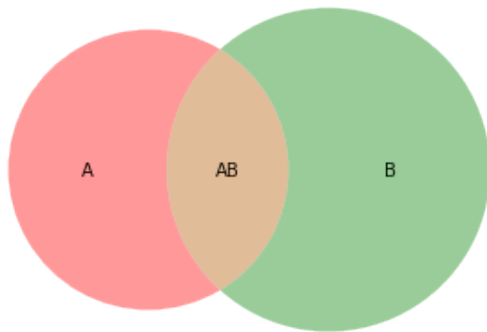
在投骰子例子中，我们让事件 A 为“出现点数为 $\{3, 4, 5\}$ 中的一个”，事件 B 为“出现点数为 $\{2, 3, 4\}$ 中的一个”。那么已知事件 B 发生，事件 A 发生的概率是多少呢？我们可以计算：

- $\Pr(A \cap B) = \Pr(\{\text{出现点数为 } 3 \text{ 或者 } 4\}) = 1/3$
- $\Pr(B) = \Pr(\text{出现点数为 } \{2, 3, 4\} \text{ 的一个}) = 1/2$

因此我们有 $\Pr(A | B) = 2/3$ ，也即在已知出现点数为 $\{2, 3, 4\}$ 中一个数的情况下，发生点数其实是 $\{3, 4, 5\}$ 中一个数的概率为 $2/3$ 。

根据韦恩 (venn) 图, 你可以理解条件概率 $\Pr(A | B)$ 为在圆 B 的范围内发生事件 A 的可能性, 即 AB 的面积除以 B 的面积。

```
import matplotlib.pyplot as plt
#下面的matplotlib_venn包需要在anaconda prompt中通过
#命令行pip install matplotlib-venn进行安装
from matplotlib_venn import venn2
v = venn2(subsets=(2,3,1), set_labels=('A', 'B'))
#参数subsets中三个元素分别代表韦恩图中A\B,B\A,AB的面积
v.get_label_by_id('10').set_text('A')
v.get_label_by_id('11').set_text('AB')
v.get_label_by_id('01').set_text('B')
```



下面介绍两个关于条件概率的例子，这两个例子都介绍了一个著名的悖论，叫做辛普森悖论。请注意，这里的辛普森并不是之前介绍的橄榄球运动员。

英国统计学家辛普森 (E.H. Simpson) 1951 年发表的一篇论文中指出，在某些情况下，在分组比较中占优势的一方，可能在合并后的总评中成为失势的一方。这个现象后来称为辛普森悖论。注意，这个辛普森是个英国统计学家，不是上述辛普森案件中的美国橄榄球明星辛普森，辛普森案件中的辛普森是 O.J.Simpson。

治愈率悖论

假设有一个人生病了，他打算从两家医院中选择一家去进行治疗。这两家医院的情况是：医院 A 的治愈率为 90%，医院 B 的治愈率为 80%。那么应该选择哪一家呢？表面上看起来，我们肯定选择医院 A，因为治愈率高嘛。但如果我们进一步看看这两家医院的统计数据，我们的决定会改变吗？

医院 A	治愈数	总数	治愈率
严重	30	100	30%
不严重	870	900	96.7%
合计	900	1000	90%

医院 B	治愈数	总数	治愈率
严重	210	400	52.5%
不严重	590	600	98.3%
合计	800	1000	80%

注意到,

$$\text{pr}(\text{医院 } A \text{ 能治愈} \mid \text{不严重}) = 0.3 < \text{pr}(\text{医院 } B \text{ 能治愈} \mid \text{不严重}) = 0.525$$

$$\text{pr}(\text{医院 } A \text{ 能治愈} \mid \text{严重}) = 0.967 < \text{pr}(\text{医院 } B \text{ 能治愈} \mid \text{严重}) = 0.983.$$

可以看到, 尽管总体来说医院 A 的治愈率要高, 但是当给定病人是否严重的情况下, 医院 B 的治愈率却总高于医院 A 。这个现象就是辛普森悖论, 即对于同一组数据, 整体的趋势和分组后的趋势表现出完全不一致的结论。这个例子中, 造成辛普森悖论的一个重要的原因是: 医院 A 和医院 B 中 “严重” 病人组与 “不严重” 病人组的比例相差较大。当两家医院具有相同的 “严重” 与 “不严重” 人数比例时, 如果医院 B 在每一组的治愈率都大于医院 A 的话, 那么医院 B 的总体治愈率肯定大于医院 A 。

美国航空协会每年出版一部《准点率汇总》，调查 30 个入选机场中，航班到港的误点百分率。每个航空公司都有其“中心区”或“空中十字路口”——航空网络的枢纽，航线由此出发呈放射状向四面八方分布。西美航空的枢纽是亚利桑那州的菲尼克斯，那里的天空很蓝。过去，30 个大型机场中，较小的阿拉斯加航空公司只能飞往其中的 5 个。其位于美国中心地区的航空枢纽是西雅图，位于最西北端，是一个真正的多雾之地。

表：5 机场误点数据

	阿拉斯加航空		西美航空	
	航班数	误点率/%	航班	误点率/%
洛杉矶	559	11.1	811	14.4
菲尼克斯	233	5.2	5255	7.9
圣迭戈	232	8.6	448	14.5
旧金山	605	16.9	449	28.7
西雅图	2146	14.2	262	23.3
总计	3775	13.3	7225	10.9

从总计看来，阿拉斯加航空的误点率都是高于西美航空的；但是，从五个不同的机场来看，阿拉斯加航空的误点率却是明显低于西美航空的。

录取率悖论

加州大学伯克利分校 1973 年秋季研究生入学数据显示，在总共 12673 名申请入学的学生中（男性 8442，女性 4321），男生的录取率是 44%，对女生的录取率是 35%。看上去，对女生似乎不公平，录取率更低。但如果按照不同院系分开看，其统计结果如下：

院系	男生		女生	
	申请人数	录取比例	申请人数	录取比例
A	825	62%	108	82%
B	560	63%	25	68%
C	325	37%	593	34%
D	417	33%	375	35%
E	191	28%	393	24%
F	373	6%	341	7%
Total	8442	44%	4321	35%

可以看到，6 个院系中有 4 个院系女生的录取率大于男生，可以说，加州大学伯克利分校更倾向于对女生的录取。

按照我们现行统计制度，国内某城市在限购令前、后商品房平均价格，2011 年比 2010 年每平米下降了 250 元，下降了 2%。

某市 2010、2011 年商品房成交数据

年份	成交套数 (套)	成交面积 (平米)	成交均价 (元/平米)	成交总额 (亿元)
2010	1000	100000	12500	12.5
2011	800	80000	12250	9.8

2010 年商品房销售地区主要是市区，2011 年市区的房源主要是郊区。

某市 2010、2011 年分市区、郊区商品房成交数据

年份	成交套数 (套)			成交均价 (万元/平米)			成交总额 (亿元)
	市区	郊区	合计	市区	郊区	合计	
2010	500	500	1000	1.5	1	1.25	12.5
2011	200	600	800	1.6	1.1	1.225	9.8

从数学上可以这么来看，即使

$$\frac{b}{a} \geq \frac{f}{e}, \quad \frac{d}{c} \geq \frac{h}{g}$$

但这不能保证

$$\frac{(b+d)}{(a+c)} \geq \frac{(f+h)}{(e+g)}.$$

条件概率是统计学中的一个核心概念，在上面的例子中起着关键的描述作用。需要注意的是，我们不对空事件取条件，因此，条件概率公式中，分母的取值范围是 $(0, 1]$ ，即不可能取到 0，从而可以看出条件概率 $\Pr(A | B)$ 大于或等于联合概率 $\Pr(AB)$ 。为什么要除以 $\Pr(B)$ ？因为在已经知道事件 B 发生的条件下，我们考虑的可能性空间就缩小到了事件 B 发生的范围内，然后将该范围内 A 发生的可能性除以 B 发生的可能性便得到了条件概率 $\Pr(A | B)$ 。

另外，需要强调的是，所有的概率本质上都是条件概率。比如，概率可以看成是给定全事件后的条件概率。不管我们是否留意，再谈到概率时总是存在这样或那样的模型、假设或条件。比如我们讨论抛硬币正面朝上的概率为二分之一，取决于我们对硬币无偏性（材质均匀等）的假设。在理解概率、条件概率的含义之后，接下来我们讨论贝叶斯公式。

为了介绍贝叶斯公式，我们首先通过一个“曲奇饼问题”的例子来引入贝叶斯公式中的相关概念。假设有两碗曲奇饼，碗 1 包含 30 个香草曲奇饼和 10 个巧克力曲奇饼，碗 2 有上述两种饼干各 20 个。

```
df = pd.DataFrame({'碗1':{'香草曲奇饼':30,'巧克力曲奇饼':10},  
                  '碗2':{'香草曲奇饼':20,'巧克力曲奇饼':20}})  
df
```

	碗 1	碗 2
巧克力曲奇饼	10	20
香草曲奇饼	30	20

现在设想你在不看的情况下随机的挑一个碗并从中随机拿一块曲奇饼，得到了一块香草曲奇饼。我们的问题是：这个香草曲奇饼来自于碗 1 的概率是多少？这就是一个条件概率问题，我们希望计算概率 $\Pr(\text{碗 1} \mid \text{香草})$ ，但怎样计算并非显而易见。注意到，若求出在碗 1 中取得香草曲奇饼的概率，则便简单得多， $\Pr(\text{香草} \mid \text{碗 1}) = 30/40 = 3/4$ 。不幸的是， $\Pr(\text{碗 1} \mid \text{香草})$ 和 $\Pr(\text{香草} \mid \text{碗 1})$ 并不相等，但是贝叶斯公式可以帮助我们通过容易计算的 $\Pr(\text{香草} \mid \text{碗 1})$ 和 $\Pr(\text{香草} \mid \text{碗 2})$ 来得到 $\Pr(\text{碗 1} \mid \text{香草})$ 。

考虑两个互不相交的事件 A_1 和 A_2 , 它们的并构成全事件 Ω 。比如, 在这个例子中, A_1 表示抽到碗 1, A_2 表示抽到碗 2。我们用事件 B 表示我们观察到的事件, 即取到香草奇饼。那么贝叶斯公式给出了如何计算 $\Pr(A_1 | B)$,

$$\Pr(A_1 | B) = \frac{\Pr(A_1)\Pr(B | A_1)}{\Pr(A_1)\Pr(B | A_1) + \Pr(A_2)\Pr(B | A_2)}.$$

其中, 右边的每个概率都能够很轻松地算出。

我们可以通过条件概率的定义及全概率公式推导出它。利用这个公式解决曲奇饼问题：

- $\Pr(A_1)$: 这是我们随机选中碗 1 的概率。由于选择碗的过程是公平且随机的，其值为 $1/2$ 。
- $\Pr(B | A_1)$: 这是从碗 1 得到一个香草曲奇饼的概率，等于 $3/4$ 。
- $\Pr(B | A_2)$: 这是从碗 2 得到一个香草曲奇饼的概率，等于 $1/2$ 。

因此根据贝叶斯公式可以算出：

$$\Pr(A_1 | B) = \frac{3}{5}.$$

说明这块香草曲奇饼来自于碗 1 的概率为 0.6，即来自碗 1 比来自碗 2 的可能性更大一点。

这里我们讨论了两个互不相交事件 A_1 和 A_2 组成了全事件 Ω 。一般地, 对于 K 个互不相交的事件 $A_1, \dots, A_K$, 且它们可以组成全事件 Ω , 那么对于某一事件 B , 我们有

$$\Pr(A_1 | B) = \Pr(A_1)\Pr(B | A_1) \bigg/ \sum_{k=1}^K \Pr(A_k)\Pr(B | A_k).$$

这个公式是由英格兰数学家贝叶斯 (Thomas Bayes, 1702-1761) 提出, 因此叫做贝叶斯公式。其中, 我们将 $\Pr(A_k)$ 叫做先验概率 (prior probability), 在事件 B 发生后, $\Pr(A_k | B)$ 叫做后验概率 (posterior probability)。

我们再来考虑一个例子，在人群中，染上某种疾病的概率是 0.1%，如果某个人在医院诊断出现了阳性，即诊断认为他患了病。假设这种诊断将实际患病人正确诊断为患病的概率是 80%，将正常人错误诊断为患病的概率是 0.05%，请问，呈阳性的结果是否可靠？

我们用事件 A_1 表示随机抽取一个人是真实患者，事件 A_2 表示随机抽取一个人是正常人，所以 $\Pr(A_1) = 0.1\%$ ， $\Pr(A_2) = 1 - \Pr(A_1)$ 。假设用事件 B 表示诊断结果呈阳性。那么诊断的功效和错误率，可以用条件概率来描述，即 $\Pr(B | A_1) = 80\%$ ， $\Pr(B | A_2) = 0.05\%$ 。紧接着，我们的目标是为了计算诊断呈阳性但实际并没有患病的概率，即 $\Pr(A_2 | B)$ 。计算这样的概率，我们求助于贝叶斯公式：

$$\Pr(A_2 | B) = \frac{\Pr(B | A_2)\Pr(A_2)}{\Pr(B | A_1)\Pr(A_1) + \Pr(B | A_2)\Pr(A_2)} \quad (2)$$

$$0.0005 * (1 - 0.001) / (0.0005*(1-0.001) + 0.8*0.001)$$

0.3843786071565987

你可以看到，虽然诊断结果是阳性，但是其为正常人的概率却很高，有大约 38%。因此，若某次在校医院体检，对于呈阳性的结果并不要慌张，这较大可能是由于错误诊断导致，你仍然是正常的。

M&M 是有各种颜色的糖果巧克力豆。制造 M&M 豆的 Mars 公司会不时变更不同颜色巧克力豆之间的混合比例。

1995 年，他们推出了蓝色的 M&M 豆。在此前一袋普通的 M&M 豆中，颜色的搭配为：30% 褐色，20% 黄色，20% 红色，10% 绿色，10% 橙色，10% 黄褐色。这之后变成了 24% 蓝色，20% 绿色，16% 橙色，14% 黄色，13% 红色，13% 褐色，也即取消了黄褐色 M&M 豆。假设我的朋友有两袋 M&M 豆，他告诉我一袋是 1994 年的，一袋是 1996 年的。但他没告诉我具体哪个袋子是哪一年的。现在他从每个袋子里各取了一个 M&M 豆给我。一个是黄色的，一个是绿色的，记作事件 B 。那么，黄色豆来自 1994 年的袋子的概率是多少？

这个问题类似于曲奇饼问题，只是变化了抽取样品的方式（碗和袋的区别）。第一步是枚举所有假设，取出黄色 M&M 豆的袋子称为袋 1，另一个称为袋 2，所以假设是

- 假设 1（记作事件 A_1 ）：袋 1 是 1994 年的，袋 2 是 1996 年的。
- 假设 2(记作事件 A_2)：袋 1 是 1996 年的，袋 2 是 1994 年的。

接着，我们设计一个表格，每行表示每个假设。每列表示贝叶斯公式中的每一项：

	先验概率 $\Pr(A)$	$\Pr(B A)$	$\Pr(A)\Pr(B A)$	后验概率 $\Pr(A B)$
假设 1	1/2	0.2×0.2	0.02	20/27
假设 2	1/2	0.14×0.1	0.007	7/27

第一列表示先验概率，基于问题的声明，两个假设的概率是相等的，即各 1/2。第二列表示似然度，表明了问题的背景信息。举例来说，如果假设 1 为真，黄色 M&M 豆问题是来自 1994 年的袋概率 20%，而绿色来自 1996 年的概率为 20%。因为选择是独立的，所以其相乘以得到 $\Pr(B | A_1)$ 。第三列由前两列相乘得到。为了得到最后一列的后验概率，我们将第三列的值归一化后得到第四列的值。因此可以看出，袋 1 是 1994 年的和袋 2 是 1996 年的概率更高。

Monty Hall 问题

Monty Hall 问题曾经是历史上最有争议的概率问题。问题看似简单，但正确答案似乎有悖常理导致很多人不能接受。Monty Hall 是游戏节目“来做个交易”的场地。

Monty Hall 问题就出自于这个节目。游戏规则是这样的，假如你是一名幸运观众，有免费拿走一辆奔驰的机会。在你面前有三扇门，其中两扇门之后是羚羊，一扇门之后是奔驰车。当你选中了一扇门后，主持人会打开剩下两扇门中是羚羊的一扇门。现在，你有两扇关着的门，主持人让你选择是否更换你的选择。那你是“换”还是“不换”呢？

大多数人都由强烈的直觉，认为这没有区别。剩下两个门没有打开，车在两扇门后的概率应该是相等的。但是，事实上如果你坚持最初的选择，你的中奖概率只有 $1/3$ ，而如果更换你选中的门，你的机会将是 $2/3$ 。

下面运用贝叶斯公式来解释这个结论。首先，我们应该对数据进行仔细地描述。在本例中，假设这三个门分别记作 a、b、c，你选择的门是 a；发生事件 B 表示主持人打开了门 b，而且没有车在后面。定义三个假设，分别记为 A_k ，假设 1(A_1)，车在 a 门后面；假设 2(A_2)，车在 b 门后面；假设 3(A_3)，车在 c 门后面。因此我们有：

	$\Pr(A_k)$	$\Pr(B \mid A_k)$	$\Pr(A)\Pr(B \mid A_k)$	$\Pr(A_k \mid B)$
车在门 a 后	1/3	1/2	1/6	1/3
车在门 b 后	1/3	0	0	0
车在门 c 后	1/3	1	1/3	2/3

定义先验很容易，我们看看似然度的计算过程：

- 考虑假设 1，如果汽车实际上是在门 a 后面，主持人可以安全的随意打开门 b 或门 c。他选择门 b 的概率为 $1/2$ 。
- 考虑假设 2，如果汽车实际上是在门 b 后，主持人不得不打开 c 门，这样他打开门 b 的概率是 0。
- 考虑假设 3，如果车在门 c 后，主持人打开门 b 的概率是 1，门 b 后无车的概率是 1。

同理，我们可以计算出每个假设在发生事件 B 后的后验概率，所以结论是最好换掉最初的选择。

该问题有许多变形。贝叶斯方法的优势之一就是可以推广到这些变形问题的处理上。比如，若主持人总是尽可能选择门 b, 且只有迫不得已的时候才选择门 c, 这种情况下，表格数据修改为：

	$\Pr(A_k)$	$\Pr(B A_k)$	$\Pr(A_k)\Pr(B A_k)$	$\Pr(A_k B)$
车在门 a 后	1/3	1	1/3	1/2
车在门 b 后	1/3	0	0	0
车在门 c 后	1/3	1	1/3	1/2

唯一的变化是如果车在门 a 后，主持人只能打开门 b。因此在这种变形问题下，改变最初的选择与否对结果的影响变得无关紧要。

随机变量 (random variable) 用来描述某个未发生事件的结果。比如从中国人民大学中选出一位学生，测量其血压，我们可以用随机变量 X 进行表示。由于测量过程收到测量误差的影响， X 总是在未知的、真实的血压附近波动，因此具有随机性，这也是随机变量的“随机”的由来。血压的随机性，到底是怎样的随机性？我们使用概率分布 (probability distribution) 来刻画。

一个简单的概率分布的例子是伯努利分布 (Bernoulli distribution)：掷一枚硬币，用随机变量 X 表述其结果： $X = 1$ 表示正面朝上， $X = 0$ 表示背面朝上。那么， $\Pr(X = 1)$ 就表达了正面朝上的概率。在我们测量血压的例子中， X 是连续型的，怎么描述它的分布呢？我们现简单介绍下经常用到的正态分布 (normal distribution)，我们将在后面一章里着重介绍它。

正态分布的形态是一个钟型曲线，它表示我们对血压的测量值由于随机误差的影响不会恰巧取得真实值（峰值），而是大概率在真实值的附近，也不会偏得太厉害。这样的钟型曲线，我们称之为正态分布的概率密度函数 (probability density function, pdf)。在数学上，我们有如下表达式。

$$pdf(x|\mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \quad (3)$$

上式中， μ 和 σ 是正态分布的两个参数。第一个参数 μ 是该分布的均值，其取值范围是任意实数，第二个参数是 σ 是标准差，用来衡量分布的离散程度，其取值只能为正。

下面我们用 Python 将正态分布 pdf 在各种参数取值下描绘出来。

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

#scipy是用于数学、工程领域的常用库
#其中scipy.stats包含了很多统计领域用到的函数
from scipy import stats

#seaborn是一个基于matplotlib的可视化python包
import seaborn as sns

mu_params = [-1,0,1]
sd_params = [0.5,1,1.5]
x = np.linspace(-7,7,100)

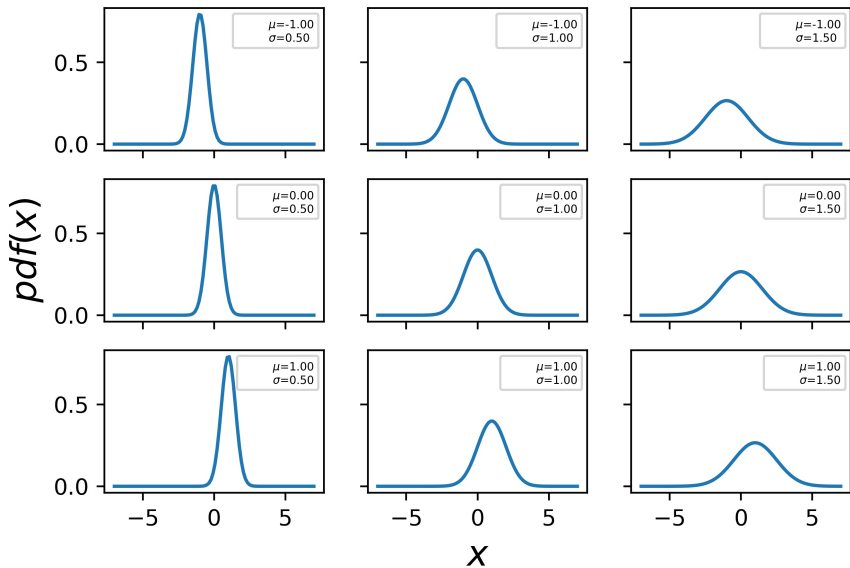
#最后两个参数表示所有子图共享同样的x轴、y轴
f,ax = plt.subplots(len(mu_params),len(sd_params),sharex=True,sharey=True)
```

正态分布 pdf

```
for i in range(3):
    for j in range(3):
        mu = mu_params[i]
        sd = sd_params[j]
        y = stats.norm(mu,sd).pdf(x)
        ax[i, j].plot(x,y)
        ax[i, j].plot(0,0, label="$\mu$={:3.2f}\n$\sigma$={:3.2f}".format(mu,sd),alpha=0)
        ax[i, j].legend(fontsize=5)

ax[2,1].set_xlabel('$x$', fontsize=16)
ax[1,0].set_ylabel('$pdf(x)$', fontsize=16)

#plt.savefig将图片保存, dpi调整清晰度
plt.savefig('fig_normal_pdf.jpg', dpi = 500)
plt.show()
```



我们可以看到，在不同的 μ 和 σ 下，正态钟型曲线具有不同的高矮胖瘦形态。 μ 刻画了正态随机变量的均值， μ 越大，钟型曲线就往右移；同时， σ 刻画了正态随机变量的波动范围， σ 越大，曲线就越矮胖，反之， σ 越小曲线就越瘦高。通常，如果一个随机变量 X 服从在参数 μ 和 σ 下的正态分布，我们可以这样表示该变量：

$$X \sim N(\mu, \sigma^2), \quad (4)$$

其中，符号 $\sim$ 读作服从于某种分布。

一般地，随机变量分为连续变量和离散变量。连续随机变量可以从某个区间内取任意值，取值是不可数的，比如正态随机变量；而离散随机变量只能取某些特定的离散值，取值是有限或可数的，比如伯努利随机变量。我们再介绍下，对于多个变量，如何描述它们的关系。如果两个随机变量 X 和 Y 对于所有可能的取值都满足 $pdf(x, y) = pdf(x)pdf(y)$ ，那么称这两个随机变量相互独立 (independent)。独立同分布 (independent and identically distributed, iid) 是指对于服从同一个分布的多个随机变量进行连续采样，各个变量的采样值之间相互独立。

在上面的例子中，我们用到 python 中的 scipy 库，scipy 服务于数学、科学、工程等领域的大量科学计算。在本章中，我们主要使用 scipy 的 stats 子模块里随机性方面的函数。

scipy.stats 中的离散分布有 13 个，连续分布有 94 个，多元分布有 8 个。针对每个分布都可以得到其（连续变量的）概率密度函数（pdf）或者（离散变量的）概率质量函数（probability mass function, pmf）、累积分布函数（cumulative distribution function, cdf）、分位数（percent point function, ppf）、随机数（random variables, rvs）、对数概率密度函数（logpdf）、对数累积分布函数（logcdf）、生存函数（survival function, sf）、均值（mean）、方差（variance）、偏度（skewness）、峰度（kurtosis）和非中心矩等。

下面是就标准正态分布对上述一些函数作图的例子。

```
plt.figure(figsize=(10,9))
x=np.arange(-4, 4, 0.01)

#做出累积分布函数
plt.subplot(2,2,1)
plt.plot(x,stats.norm.cdf(x))
plt.title('cdf of  $N(0,1)$ :  $\Phi(x)$ ')

#做出概率密度函数
plt.subplot(2,2,2)
plt.plot(x, stats.norm.pdf(x))
plt.title('pdf of  $N(0,1)$ :  $\phi(x)$ ')
```

作图

#做出生存函数

```
plt.subplot(2,2,3)
```

```
plt.plot(x, stats.norm.sf(x))
```

```
plt.title('sf of  $N(0,1)$ :  $1-\Phi(x)$ ')
```

#做出逆累积分布函数

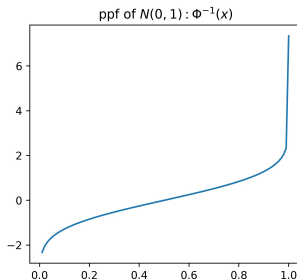
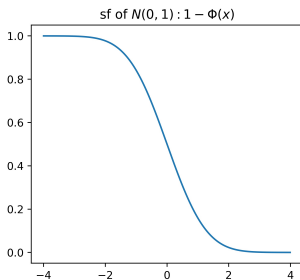
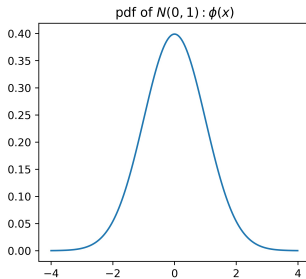
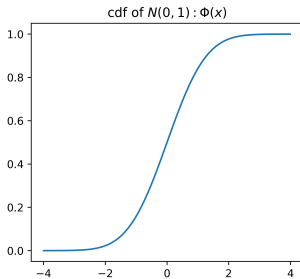
```
plt.subplot(2,2,4)
```

```
plt.plot(x, stats.norm.ppf(x))
```

```
plt.title('ppf of  $N(0,1)$ :  $\Phi^{-1}(x)$ ')
```

```
plt.savefig('figs_normal.jpg', dpi = 500)
```

```
plt.show()
```



下面来看如何产生 10 个均值为 5，标准差为 2 的正态分布的随机数。

```
#固定随机种子1010,以重现结果  
np.random.seed(1010)  
stats.norm.rvs(size=10, loc=5, scale=2)
```

```
array([2.6491042 , 4.23370464, 2.05726764, 1.39886295, 5.26020084,  
      8.19123725, 6.98632135, 0.27258561, 4.04081547, 1.69923611])
```

我们也可以在函数 `stats.norm.rvs` 中设定随机种子。

```
stats.norm.rvs(size=10, random_state=1010, loc=5, scale=2)
```

```
array([2.6491042 , 4.23370464, 2.05726764, 1.39886295, 5.26020084,  
      8.19123725, 6.98632135, 0.27258561, 4.04081547, 1.69923611])
```

请注意，代码 `stats.norm.rvs(10)` 会产生一个服从均值为 10、标准差为 1 的随机数，而 `stats.norm.rvs(size=10)` 产生 10 个服从标准正态的随机数。如果反复使用某个分布，就可以把它赋值给一个对象，然后就不用每次都输入那些涉及分布及参数选项的代码了。

```
fr = stats.norm(loc=5, scale=2)
print(fr.rvs(size=5, random_state=1010))
print(fr.mean(), fr.std(), fr.cdf(5.97), fr.pdf([-0.5, 2.96]),
      fr.stats(moments='mvsk'), fr.kwds)
```

```
[2.6491042 4.23370464 2.05726764 1.39886295 5.26020084]
5.0 2.0 0.6861618272430887 [0.00454678 0.11856598] (array(5.), array(4.), array(0.),
array(0.))
{'loc': 5, 'scale': 2}
```

注意这里 “mvsk” 分别指均值、方差、偏度和峰度，默认值是 “mv”。下面的例子是产生一些概率点的标准正态分布右侧尾临界值表。

```
stats.norm.isf([0.1, 0.05, 0.025, 0.01, 0.001])
```

```
array([1.28155157, 1.64485363, 1.95996398, 2.32634787, 3.09023231])
```

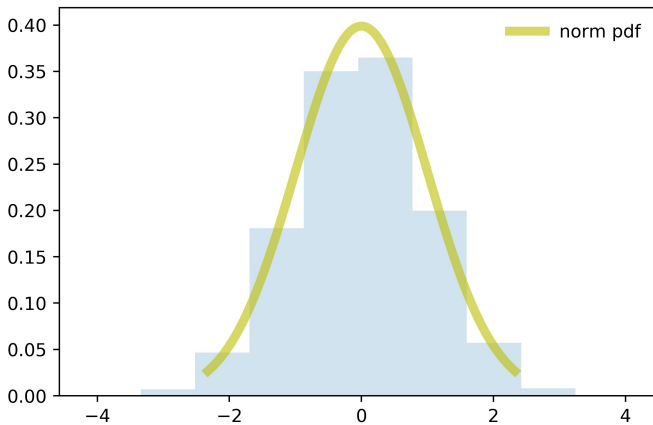
```
-stats.norm.ppf([0.1, 0.05, 0.025, 0.01, 0.001])
```

```
array([1.28155157, 1.64485363, 1.95996398, 2.32634787, 3.09023231])
```

下面我们通过从正态分布中抽出 10000 个样本来画出正态分布的概率密度函数。

```
from scipy.stats import norm
fig, ax = plt.subplots(1, 1)

x = np.linspace(norm.ppf(0.01), norm.ppf(0.99), 100)
ax.plot(x, norm.pdf(x), 'y-', lw=5, alpha=0.6, label='norm pdf')
r = norm.rvs(size=10000)
ax.hist(r, density=True, histtype='stepfilled', alpha=0.2)
ax.legend(loc='best', frameon=False)
plt.savefig('hist_normal.jpg', dpi = 500)
plt.show()
```



numpy 模块中随机数的产生

除了用 `scipy.stats` 模块生成很多分布的随机数外，我们也可以利用 `numpy` 的 `random` 子模块。比如，`np.random.random()` 用于生成 (0,1) 中的随机符点数。

```
np.random.random(5)
```

```
array([0.67882587, 0.99915303, 0.57957621, 0.58361583, 0.56117458])
```

#生成的随机数放置在一个2乘以3的矩阵中

```
np.random.random([2, 3])
```

#也可以写成 `np.random.random((2,3))`

```
array([[0.57225548, 0.97102695, 0.89805356],  
       [0.27749992, 0.80221771, 0.08758212]])
```

一般地, `np.random.random((d0, d1, ..., dn))` 可以返回一个 `d0` 乘以 `d1` 一直到 `dn` 的随机样本阵列。随机数是来自于 $[0,1)$ 的均匀分布, 它和 `np.random.rand(d0, d1, ..., dn)` 具有相同的功能。

```
np.random.rand(2,2)
```

```
array([[0.12270552, 0.54859833],  
       [0.0103439 , 0.33442629]])
```

`np.random.randn(d0, d1, ..., dn)` 生成一个 `d0` 乘以 `d1` 一直到 `dn` 的来自标准正态分布的随机样本阵列。其输入通常为整数，但是如果为浮点数，则会自动直接截断转换为整数。

```
np.random.randn(2, 3)
```

```
array([[ 1.44886002,  0.68391819, -0.75206405],  
       [-1.58914272, -1.12089084, -0.56034968]])
```

`np.random.standard_normal` 函数与 `np.random.randn(6)` 具有相似的功能，通过下面例子你可以看到他们的微妙差别。

```
np.random.standard_normal((2, 5))
```

```
array([[ 2.26298763, -0.0186085 , -0.20468903, -0.81302621, -1.66655772],  
       [ 1.23609895, -1.56535411, -0.53312755, -1.41671357, -1.05268034]])
```

`np.random.standard_normal()` 函数与 `np.random.randn()` 类似，但是 `np.random.standard_normal()` 的输入参数为元组 (tuple)；`np.random.randn()` 的输入通常为整数，但是如果为浮点数，则会自动直接截断转换为整数。

`np.random.randint(low, high=None, size=None, dtype='i')`: 生成一个整数或 n 维整数数组，取数范围：若 `high` 不为 `None` 时，取 $[low, high)$ 之间随机整数，否则取值 $[0, low)$ 之间随机整数。`dtype` 选择输出的格式，如 `int32`、`int64`、`uint8` 等（使用 `dtype` 函数查看数据类型）。

```
#从3到9中随机抽取整数，排列成一个2乘以5的矩阵
np.random.randint(3,10,size=(2,5))
```

```
array([[6, 5, 5, 6, 4],
       [5, 4, 9, 8, 9]])
```

```
d = np.random.randint(3,10,size=(2,5), dtype='int64')
d.dtype
```

```
dtype('int64')
```

```
numpy.random.choice(a, size=None, replace=True, p=None)
```

从序列中获取元素，若 a 为整数，元素取值为 $\text{np.arange}(a)$ 中随机数；若 a 为数组，取值为 a 数组元素中随机元素。 replace 表示是否为有放回抽样， True 表示可以抽出重复的元素，如果是 False 表示抽出的元素不放回。 p 表示每个元素被抽取的概率。

```
np.random.choice(a=5, size=3, replace=False, p=[0.2, 0.1, 0.3, 0.4, 0.0])
```

```
array([3, 0, 2])
```

```
np.random.choice(a=['d','e','f','g','h'], size=3, replace=False,  
                  p=[0.2, 0.1, 0.3, 0.4, 0.0])
```

```
array(['e', 'd', 'g'], dtype='<U1')
```

`np.random.shuffle(x)` 对 `x` 进行重排序, 如果 `x` 为多维数组, 只沿第一条轴洗牌, 输出为 `None`。

```
items = [1, 2, 3, 4, 5, 6]
np.random.shuffle(items)
items
```

`[4, 6, 1, 5, 3, 2]`

`np.random.permutation(x)` 与 `np.random.shuffle(x)` 函数功能类似, 请注意两者区别是 `permutation(x)` 不会修改 `x` 的顺序。

```
item1 = np.array([1, 2, 3, 4, 5, 6])
item2 = np.random.permutation(item1)
print(item1)
print(item2)
```

`[1 2 3 4 5 6]`

`[5 6 1 3 4 2]`